

Deep Research: Avatrada 57 Topic 022

Engineering Report: Fractional Kelly Algorithm

Authored By: Autonomous Technical Researcher **Date:** October 26, 2023
Subject: Deep-Dive on the Fractional Kelly Sizing Algorithm

Executive Summary: This report provides a detailed analysis of the Fractional Kelly Algorithm as specified in the source document. The Kelly Criterion is a mathematical formula for bet sizing that seeks to maximize the long-term geometric growth rate of capital. The specified implementation uses a "Quarter Kelly" approach, scaling the theoretically optimal fraction by `0.25` to produce a more conservative position size. This analysis deconstructs the formula, provides a robust Python implementation within a `KellySizer` class, and critically examines the algorithm's potential failure modes, edge cases, and practical considerations, including its application to short selling.

1. Technical Deconstruction

The core of the system is the Kelly Criterion formula, modified by a fractional scalar. We will break down each component.

1.1. The Kelly Criterion Formula

The standard Kelly Criterion formula for a binary outcome (win/loss) is:

$$f = p - q/b$$

Where: - f is the optimal fraction of capital to risk. - p is the probability of a win (the Win Rate). - q is the probability of a loss ($1 - p$). - b is the payoff ratio (net profit on a win / net loss on a loss).

The formula provided in the source document is an algebraic equivalent:

$$f^* = \frac{p(b+1)-1}{b}$$

Derivation of Equivalence: 1. Start with $f = p - (1-p)/b$ 2. Find a common denominator: $f = (p*b)/b - (1-p)/b$ 3. Combine terms: $f = (pb - 1 + p) / b$ 4. Factor out p : $f = (p(b+1) - 1) / b$

This confirms the provided formula is a standard representation of the Kelly Criterion.

1.2. System Parameters

The algorithm's behavior is dictated by three key inputs, which are recalibrated every 90 days:

- **p (Win Rate):** The historical probability of a trade being profitable. It is a value between 0 and 1.
- **b (Payoff Ratio):** This is the ratio of the average win size to the average loss size. It is derived from the prompt's inputs: $b = \frac{\text{avg_win}}{\text{avg_loss}}$ A b value of 2.0 means that the average winning trade is twice as large as the average losing trade.
- **Fractional Scalar (0.25):** This is a crucial risk management overlay. The "Full Kelly" fraction (f^*) is known to be highly aggressive and sensitive to estimation errors in p and b . Multiplying by a scalar (like 0.25 for "Quarter Kelly") significantly reduces risk and volatility, protecting against the inevitable inaccuracies of forecasting future returns from past data.

1.3. Core Mechanism

The algorithm's objective is to determine the optimal fraction of capital to allocate to a given strategy. The core logic relies on the **positive expectancy** of

the strategy. The numerator, $p(b+1) - 1$, is directly related to the strategy's "edge" or expected value (EV).

- **Positive Expectancy (Edge):** A strategy is only worth pursuing if its expected value is positive. $EV = (p * avg_win) - ((1-p) * avg_loss) > 0$
If $EV \leq 0$, the Kelly fraction will be zero or negative, correctly indicating that no capital should be risked. The implementation must include a safeguard for this.
-

2. Implementation Strategy

This section details the construction of the `KellySizer` class in Python, adhering to the prompt's requirements.

2.1. Libraries and Patterns

- **Language:** Python 3.x
- **Libraries:** No external libraries are required. Standard math operations are sufficient.
- **Pattern:** An object-oriented approach using a class (`KellySizer`) is ideal. This encapsulates the logic and state (win rate, payoffs, scalar) cleanly, allowing for easy instantiation and reuse for different trading strategies.

2.2. Python Class: `KellySizer`

The following Python code provides a complete implementation of the `KellySizer` class.

```
import math

class KellySizer:
    """
    Calculates the optimal position size using the Fractional Kelly Criterion.

    This class takes the statistical properties of a trading strategy (win rate,
    average win, and average loss) and computes the fraction of capital to
```

```

        allocate per trade, scaled by a conservative factor.
    """

    def __init__(self, win_rate: float, avg_win: float, avg_loss: float,
kelly_fraction: float = 0.25):
        """
        Initializes the KellySizer with strategy parameters.

        Args:
            win_rate (float): The probability of a winning trade (e.g., 0.6 for
60%).
            avg_win (float): The average profit from a winning trade (must be >
0).
            avg_loss (float): The average loss from a losing trade (must be >
0).
            kelly_fraction (float): The scalar to apply to the full Kelly
fraction.
                                Defaults to 0.25 (Quarter Kelly).
        """
        if not (0 <= win_rate <= 1):
            raise ValueError("win_rate must be between 0 and 1.")
        if avg_win <= 0:
            raise ValueError("avg_win must be positive.")
        if avg_loss <= 0:
            raise ValueError("avg_loss must be positive to avoid division by
zero.")
        if not (0 < kelly_fraction <= 1):
            raise ValueError("kelly_fraction must be between 0 and 1.")

        self.p = win_rate
        self.avg_win = avg_win
        self.avg_loss = avg_loss
        self.fractional_scalar = kelly_fraction

        # Calculate the payoff ratio 'b'
        self.b = self.avg_win / self.avg_loss

    def calculate_fraction(self) -> float:
        """
        Computes the Fractional Kelly stake.

```

```

    Includes a safeguard to return 0 if the strategy has a negative
    expected value.

    Returns:
        float: The recommended fraction of capital to risk, between 0 and 1.
    """
    # 1. Safeguard: Check for positive expected value (the "edge")
    # EV = (p * avg_win) - ((1-p) * avg_loss)
    # A simpler check is if p*b > (1-p)
    if (self.p * self.b) <= (1 - self.p):
        return 0.0

    # 2. Calculate the full Kelly fraction using the source formula
    # f = (p(b+1) - 1) / b
    full_kelly_fraction = (self.p * (self.b + 1) - 1) / self.b

    # 3. Apply the fractional scalar
    fractional_kelly = self.fractional_scalar * full_kelly_fraction

    # 4. Final safeguard: Ensure the result is non-negative
    return max(0.0, fractional_kelly)

    def __repr__(self) -> str:
        return (f"KellySizer(win_rate={self.p}, avg_win={self.avg_win}, "
                f"avg_loss={self.avg_loss},
                kelly_fraction={self.fractional_scalar})")

```

2.3. Handling Short Selling Strategies

The prompt specifically asks how to handle the 'Odds' (`b`) for short selling. The Kelly formula is agnostic to the direction of the trade (long or short). The key is to define "win" and "loss" from the perspective of the strategy.

For a **short selling strategy**: * A **"win"** occurs when the asset price *decreases*, and the position is closed for a profit. * A **"loss"** occurs when the asset price *increases*, and the position is closed for a loss.

Therefore, to use the `KellySizer` for a short strategy, you simply provide the historical statistics of that shorting strategy: * `win_rate`: The percentage of short trades that were profitable. * `avg_win`: The average profit made on the winning short trades. * `avg_loss`: The average loss incurred on the losing short trades.

The formula and the class handle it correctly without any modification.

2.4. Example Usage

```
# --- Example 1: A profitable long strategy ---
# 55% win rate, average win is 1.5x the average loss
long_strategy = KellySizer(win_rate=0.55, avg_win=1500, avg_loss=1000,
kelly_fraction=0.25)
long_fraction = long_strategy.calculate_fraction()
print(f"Long Strategy Payoff Ratio (b): {long_strategy.b:.2f}")
print(f"Recommended Fraction for Long Strategy: {long_fraction:.4f} (or
{long_fraction:.2%})")
# Expected output: ~5.13%

# --- Example 2: A profitable short strategy ---
# 65% of short trades are winners, average win is 0.8x the average loss
short_strategy = KellySizer(win_rate=0.65, avg_win=800, avg_loss=1000,
kelly_fraction=0.25)
short_fraction = short_strategy.calculate_fraction()
print(f"\nShort Strategy Payoff Ratio (b): {short_strategy.b:.2f}")
print(f"Recommended Fraction for Short Strategy: {short_fraction:.4f} (or
{short_fraction:.2%})")
# Expected output: ~4.06%

# --- Example 3: An unprofitable strategy ---
# Low win rate and poor payoff
bad_strategy = KellySizer(win_rate=0.40, avg_win=1200, avg_loss=1000)
bad_fraction = bad_strategy.calculate_fraction()
print(f"\nBad Strategy Payoff Ratio (b): {bad_strategy.b:.2f}")
print(f"Recommended Fraction for Bad Strategy: {bad_fraction:.4f} (or
{bad_fraction:.2%})")
# Expected output: 0.00% because the expected value is negative
```

3. Critical Analysis

While mathematically elegant, the Kelly Criterion has significant practical limitations and failure modes that must be understood.

3.1. Potential Failure Modes & Edge Cases

- 1. Parameter Estimation Error (Garbage In, Garbage Out):** This is the single greatest weakness. The formula's output is extremely sensitive to the inputs `p` and `b`. These are *estimates* based on historical data (e.g., the last 90 days) and are not guaranteed to persist. If the actual win rate is lower or average loss is higher than estimated, the "optimal" fraction can lead to severe drawdowns or even total ruin. The use of a fractional scalar (`0.25`) is a direct and necessary mitigation for this risk.
- 2. Non-Stationarity of Markets:** Financial markets are not static. A strategy's `p` and `b` can and will change over time due to shifts in volatility, liquidity, or market regime. The 90-day recalibration period is an attempt to adapt, but a sudden regime shift can render the current parameters obsolete and dangerous long before the next recalibration.
- 3. Assumption of Independent Bets:** The Kelly Criterion assumes that each trade is statistically independent of the others. In practice, this is rarely true. A portfolio manager might apply the same strategy to multiple correlated assets (e.g., several large-cap tech stocks). This creates hidden concentration risk, as a single market factor could cause all positions to lose simultaneously, leading to a much larger drawdown than the model predicts.
- 4. Fat Tails and Black Swans:** The model implicitly assumes a known and stable distribution of returns. It does not account for "Black Swan" events—rare, high-impact events that can cause losses far exceeding the historical `avg_loss`. A short squeeze, for example, can create theoretically infinite losses for a short seller, a risk not captured by the formula.
- 5. Division by Zero:** If `avg_loss` is zero (or infinitesimally small), the payoff ratio `b` would approach infinity, breaking the calculation. The

implementation correctly guards against this by requiring a positive `avg_loss`.

3.2. Optimizations and Enhancements

1. **Dynamic Fractional Scalar:** Instead of a fixed `0.25`, the scalar could be dynamic. For instance, it could be adjusted based on the statistical confidence in the `p` and `b` estimates. A strategy with a long track record and low variance in its metrics might justify a higher scalar (e.g., 0.4), while a new or erratic strategy might warrant a lower one (e.g., 0.1).
2. **Volatility Targeting Overlay:** The Kelly Criterion maximizes geometric growth but ignores the path taken to get there. It can produce strategies with gut-wrenching volatility and deep drawdowns. A common professional practice is to use the Kelly fraction as an input but ultimately cap the position size based on its contribution to overall portfolio volatility.
3. **Robust Estimation Techniques:** Instead of using simple historical averages for `p`, `avg_win`, and `avg_loss`, more robust statistical methods could be employed. For example, using bootstrapping to simulate thousands of possible equity curves could provide a distribution of potential Kelly fractions, allowing the user to choose a more conservative value from that range (e.g., the 25th percentile).
4. **Multi-Asset Kelly:** For portfolio-level optimization, the single-asset Kelly formula is insufficient. A multi-asset (or covariance) Kelly formula should be used, which accounts for the correlations between different assets to optimize the entire portfolio's growth rate, not just individual positions.